

5 - A-Star Search

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Combine path cost and heuristic estimate using $f(n)=g(n)+h(n)$.
- Implement A* graph search.
- Explain the significance of admissible and consistent heuristics.

Background

A* balances the cost already paid with an estimate of the cost still required. With suitable heuristic conditions, A* can return an optimal path while usually exploring far fewer states than uninformed uniform-cost search.

Core Concepts

Concept	Meaning in this lab
$g(n)$	Known cost from the start to n .
$h(n)$	Estimated cost from n to the goal.
$f(n)$	Estimated total solution cost $g(n)+h(n)$.
Admissible heuristic	Never overestimates the true remaining cost.
Consistent heuristic	Satisfies a triangle-inequality-style condition across edges.

Guided Procedure

1. Represent edges as neighbor-cost pairs.
2. Maintain a priority queue ordered by $f=g+h$.
3. Store the best g value seen for every state.
4. When a cheaper route to a state is found, update its record and push the improved entry.
5. Test with $h=0$ and observe that behavior approaches uniform-cost search.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import heapq

def astar(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        if g != best_g.get(node):
            continue
        for nxt, cost in graph.get(node, []):
            ng = g + cost
            if ng < best_g.get(nxt, float("inf")):
                best_g[nxt] = ng
                heapq.heappush(pq, (ng + h[nxt], ng, nxt, path + [nxt]))
    return None
```

Lab Exercises

Task 1. Run A* on a small road-map graph and report the chosen route and total cost.

Task 2. Repeat with $h(n)=0$ for every node and compare expansions.

Task 3. Construct a heuristic that overestimates one state and discuss the risk to optimality.

Task 4. Extension: solve the 8-puzzle using misplaced-tiles or Manhattan-distance heuristics.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why does A* use both g and h ?
2. What does admissibility mean?
3. Why can two different admissible heuristics lead to different runtimes?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.